

<https://github.com/SandyaSivakumar/Mini-Compiler>

## Team Workflow of Compiler (Stage-by-Stage Integration)

---

### 1. Manish (Lexical Analyzer – LEX Tool)

#### ♦ What this person does

- Writes **LEX (.l file)**
- Defines **regular expressions** for:
  - Keywords (int, if)
  - Identifiers ([a-zA-Z][a-zA-Z0-9]\*)
  - Numbers, operators, symbols
- Ignores spaces, comments
- Inserts variables into **Symbol Table**

#### ♦ Output given to next stage

1. **Token Stream** → passed to YACC
2. **Symbol Table** → shared globally

#### ♦ Important Integration

- Returns tokens using:

```
return TOKEN_NAME;
```

- Sends values using:

```
yylval = value;
```

#### ♦ Flow

Source Code → LEX → Tokens + Symbol Table

---

### 2. Inayat Ahmed (Syntax Analyzer – YACC Tool)

#### ♦ What this person does

- Writes **YACC (.y file)**
- Defines **grammar rules (CFG)**:

$$E \rightarrow E + T$$

$$T \rightarrow T * F$$

- Uses tokens from LEX (%token)
- Handles:
  - Operator precedence
  - Syntax errors (yyerror)

#### ♦ **Input**

- Tokens from LEX (yylex() function)

#### ♦ **Output**

1. **Parse Tree / AST**
2. Syntax validation result

#### ♦ **Important Integration**

- Calls LEX automatically:

yylex();

#### ♦ **Flow**

Tokens → YACC → AST

---

### **3. Salman (Semantic Analyzer)**

#### ♦ **What you do**

- Work on **AST generated by YACC**
- Use **Symbol Table from LEX**
- Perform:
  - Type checking
  - Scope checking
  - Function validation

#### ♦ **Input**

- AST (from YACC)
- Symbol Table (from LEX)

#### ♦ Output

1. **Annotated AST**
2. Semantic error list

#### ♦ Example Task

int a;

a = "hello"; ❌ → Type error

#### ♦ Flow

AST + Symbol Table → Semantic Analysis → Annotated AST

---

## 4. Musaib (Intermediate Code Generator)

#### ♦ What this person does

- Traverses **Annotated AST**
- Generates:
  - Three Address Code (3AC)
  - Temporary variables (t1, t2)

#### ♦ Input

- Annotated AST

#### ♦ Output

- Intermediate Code (3AC)

#### ♦ Example

a = b \* c + d

↓

t1 = b \* c

t2 = t1 + d

a = t2

- ♦ **Flow**

Annotated AST  $\rightarrow$  3AC

---

## 5. Chaithanya (Optimizer & Backend)

- ♦ **What this person does**

- Optimizes intermediate code
- Generates final output

- ♦ **Input**

- 3AC (Intermediate Code)

- ♦ **Output**

1. Optimized Code
2. Assembly / Machine Code

- ♦ **Example**

$t1 = 2 + 2 \rightarrow t1 = 4$

- ♦ **Flow**

3AC  $\rightarrow$  Optimization  $\rightarrow$  Final Code

---

## COMPLETE TEAM PIPELINE (VERY IMPORTANT)

[Manish : LEX]

Source Code



Tokens + Symbol Table



[Inayat Ahmed : YACC]

AST



[Salman 3: Semantic]

Annotated AST

↓  
[Musaib 4: IC Generator]  
3 Address Code  
↓  
[Chaithanya 5: Optimizer]  
Optimized Code / Assembly

---

♦ **Runtime Flow**

YACC calls yylex()  
↓  
LEX returns tokens  
↓  
YACC builds AST

---